IA no es chat

ES UN SISTEMA DE DECISION ASISTIDA

- Sin hype
- Sin marketing
- Sin confundir chat con producto
- Foco: resolver problemas reales

La formula que manda

FORMULA OPERATIVA

```
modelo + contexto + herramientas + memoria + workflow + evaluacion
```

- Si falta una pieza, baja la calidad
- Si faltan varias, tenes un chatbot frágil
- La IA no se "usa": se diseña

Donde se rompe

- Reducir IA a prompt engineering
- Creer que "mejor modelo" arregla mal sistema
- Pensar agentes = magia autonoma
- Medir por demo, no por confiabilidad

Del problema al sistema

DEL INPUT A LA EVALUACION

```
Problema → entradas → decision → accion → control -
> evaluacion
```

- `LLM` = una pieza del pipeline
- `Sistema` = la orquestacion completa

El modelo es una pieza

ELEGILO POR SISTEMA, NO POR FETICHE

`Capacidad` + `latencia` + `costo` + `contexto`

- Aporta razonamiento probabilistico, no verdad
- Puede ser generalista, especializado o multimodal
- El modelo mas caro no corrige mal contexto ni mal workflow

Utilidad real del modelo

- Transformar texto, codigo, imagen, audio
- Resumir, clasificar, extraer, planificar
- Generar borradores y propuestas
- Traducir entre formatos humanos y estructurados

LIMITES REALES

- Recordar hechos sin fuente confiable
- Ser consistente sin control externo
- Operar con precision contractual sin validacion

El contexto decide

- El modelo responde a lo que ve, no a lo que vos asumis
- Context engineering > prompt bonito
- Contexto es seleccion, compresion y orden
- Mas tokens no siempre significa mejor respuesta

Diseñar entrada, no adornar prompts

- Definir objetivo, restricciones y formato de salida
- Inyectar estado relevante, no ruido historico
- Separar instrucciones, datos y herramientas
- Compactar sin perder decisiones criticas

REGLA

Menos basura en contexto = menos error acumulado

Compactacion

- Conversaciones largas degradan foco y costo
- Compactar es resumir estado util, no cortar al azar
- Hay que preservar decisiones, supuestos y pendientes
- Si compactas mal, el agente "olvida" arquitectura

Prompt/contexto gigante no escala

- Meter todo el historial en la ventana mezcla señal, ruido y costo
- El prompt termina cargando instrucciones, estado, decisiones y basura historica
- Cuando todo vive en contexto, cada tarea arranca pesada y frágil
- Escalar pide separar: skill reusable, sub-tarea acotada y memoria recuperable

Memoria no es historial

- Memoria != transcript ni ventana actual
- Memoria = señal recuperable con criterio de escritura
- Se lee por necesidad: hechos, decisiones, restricciones, estado previo
- Sin politica de lectura/escritura, memoria = basura cara con latencia

Retrieval empieza antes del prompt

LA CALIDAD SE DEFINE ANTES DE CONSULTAR

ANTES DEL PROMPT	IMPACTO REAL
Chunking y metadata	Deciden que unidad se puede recuperar
Indice y filtros	Separan señal de ruido
Ranking y corte	Determinan que entra al contexto

- Si indexaste mal, el prompt solo maquilla basura

RAG sin humo

- RAG no es "meter PDFs al prompt"
- Pipeline minimo:

```
Fuente → chunking → embeddings → indice → retrieval → contexto → respuesta
```

- El cuello de botella suele ser retrieval, no generacion

Tools dan evidencia

SIN ACCION VERIFICABLE, TODO SUENA PLAUSIBLE

> `consultar → ejecutar → observar`

- Buscar docs, leer archivos o llamar APIs cambia opinion por evidencia
- Tool calling sirve cuando la salida deja rastro verificable
- Contratos claros: `input`, `output`, `errores`, `permisos`

Protocolo no arregla mala tool

- MCP ordena como exponer herramientas y recursos a modelos
- Sirve para interoperabilidad, no para "hacer magia"
- Un buen protocolo baja acoplamiento y mejora observabilidad
- Si la herramienta es mala, el protocolo no te salva

Skills y sub-agentes bajan entropia

- Skill = criterio reusable fuera de la ventana del chat
- Sub-agente = contexto acotado para una tarea puntual
- Separar tareas reduce mezcla de responsabilidades e instrucciones
- Memoria persistente guarda continuidad sin inflar cada prompt

Anti-patron: God Agent

- Un solo agente con todo el contexto, todas las tools y todos los permisos
- Planea, ejecuta, recuerda y decide sin fronteras claras
- Parece flexible en demo; en produccion mezcla errores, costo y trazas
- Si no podes separar roles o pasos, no diseñaste control: diseñaste fe

Agentes, sin misticismo

- Agente = LLM + tools + state + policy
- A veces tambien incluye scheduler, memoria y evaluador
- No es "conciencia" ni autonomia ilimitada
- Es un loop controlado con objetivos y restricciones

Loop minimo de agente

```
Objetivo
  → plan corto
  → usar herramienta
  → observar resultado
  → actualizar estado
  → decidir siguiente paso
  → finalizar o escalar
```

- Si no hay politica de corte, el agente deriva

A veces alcanza con un DAG

- No todo problema requiere un agente libre
- Muchas veces alcanza con workflow deterministico
- DAG = pasos con dependencias explicitas
- Menos autonomia suele dar mas trazabilidad

Workflow auditable antes que autonomia

```
explore → spec/design → tasks → apply → verify
```

- No es religion ni framework: es separar pensamiento, ejecucion y control
- Cada paso deja artefactos revisables y menos lugar para improvisacion
- Muchas veces esto resuelve mas que soltar un agente "general"

Sin evals no hay sistema

- Lo que no evaluas, se rompe en silencio
- Evals pueden medir exactitud, formato, seguridad y costo
- Necesitas casos felices, bordes y regresion
- Prompt sin eval es fe, no ingenieria

Guardrails no tapan deuda tecnica

- Alucinacion = salida plausible sin sustento suficiente
- Se reduce con retrieval, tools, validacion y formatos estrictos
- Guardrails sirven para limitar acciones y salidas
- Guardrail no reemplaza arquitectura ni evaluacion

Seguridad desde el diseno

- Riesgos: prompt injection, data leakage, tool abuse, exfiltracion
- Principio minimo privilegio para herramientas y secretos
- Separar datos publicos, internos y sensibles
- Registrar quien hizo que y con que contexto

Costos: mas que tokens

EL COSTO REAL APARECE EN OPERACION

CAPA	TE PEGA EN
Modelo	tokens, latencia, routing
Contexto	precision, tiempo, retries
Tools	llamadas externas, fallas, observabilidad
Sistema	cache, batching, soporte, debugging

- Optimizar solo tokens te deja ciego al costo real

No mezcles apertura con control

- Open source: codigo y licencias abiertas del software
- Open weights: pesos disponibles, no siempre con licencia abierta total
- Self-hosted: corre en tu infra, no implica apertura ni soberania completa
- Open weights != open source != privacidad garantizada

Cuando texto ya no alcanza

- Texto ya no alcanza para muchos flujos reales
- Entrada: voz, imagen, video, OCR, UI, documentos
- Salida: texto, audio, imagen, video, acciones
- El diseño cambia cuando el contexto no es solo texto

Saber cuando frenar

- No todo se debe automatizar de punta a punta
- Escalar a humano cuando hay riesgo, ambigüedad o impacto alto
- Aprobacion humana puede ser muestreo, revision o paso obligatorio
- Buen sistema sabe cuando frenar

Sin trazas no hay debug

- Loggear prompts no alcanza
- Hay que observar contexto usado, tools invocadas y resultados
- Medir latencia, errores, costo y calidad
- Sin trazas, debuggear IA es adivinanza cara

Capas minimas del sistema

SI FALTA UNA, LA FRAGILIDAD SUBE

> `capacidad → ejecucion → control`

- `Capacidad` -> modelo, contexto, memoria, retrieval
- `Ejecucion` -> herramientas, policy/workflow
- `Control` -> evaluacion, observabilidad, seguridad

Mapa de herramientas

PENSALO POR CAPA, NO POR CATALOGO

CAPA	PARA QUE SIRVE	EJEMPLOS
Modelos	razonar y generar	Claude, ChatGPT, Gemini
Coding agents	operar sobre repos	OpenCode, Claude Code
Contexto / memoria	recuperar estado	Context7, Engram
Multimedia	ampliar inputs y outputs	Remotion, GLM-OCR

- La pregunta correcta no es `cual uso`, sino `que capa me falta`

Herramientas con tradeoffs

CADA CAPA COMPRA ALGO Y PAGA OTRA COSA

OPCION	GANAS	PAGAS
Frontier APIs	calidad inicial y velocidad	dependencia y menos control
Open weights	control e infra propia	operacion, tuning y costo fijo
Memory systems	continuidad util	basura cara sin criterio
Coding agents	velocidad de entrega	deuda amplificada sin workflow
Multimedia tools	nuevos inputs y outputs	mas pipeline y timing

Orden de diseno

1. Definir problema y riesgo
2. Elegir el minimo sistema suficiente
3. Diseñar contexto, memoria y tools
4. Poner evals y observabilidad desde el inicio
5. Recién despues optimizar modelo

Sin sistema no hay IA util

EL MODELO IMPORTA. LA ARQUITECTURA DECIDE.

```
entradas / decision / herramientas / control / evaluacion
```

Si no podes dibujar eso, no diseñaste IA util: diseñaste una demo.

IA util = decision asistida con criterio

Arquitectura explicable antes que prompt vistoso